

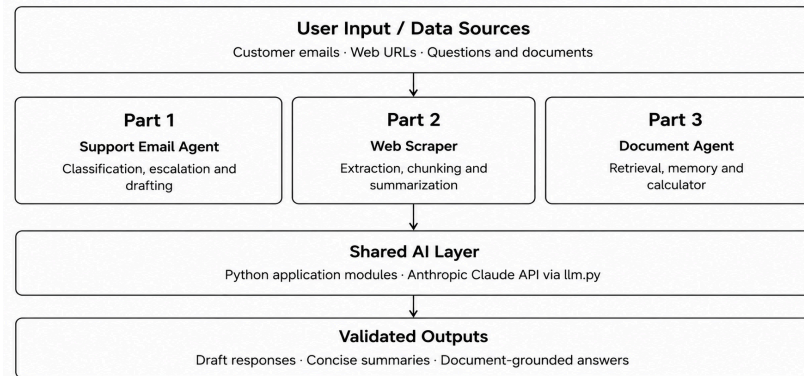
# Agentic Architect Challenge

System Architecture, Design Decisions & Failure Handling

## 1. System Overview

The project has three separate AI-driven workflows written in Python with the Anthropic Claude API. The three workflows solve unique problems: customer support automation, extraction and summarisation of documents on the web, and Q&A in the document.

## 2. System Architecture



This diagram shows the logical components. The three workflows operate independently and share the LLM integration module.

## 3. Component Design

### Part 1. Customer Support Automation

- Classifies incoming emails into Billing, Technical, Feedback, and other categories.
- Ensures that the following critical conditions are checked beforehand: Data loss, service outage, security breach, or more than three contacts in seven days.
- Raises critical cases for human review.
- Applies the knowledge base to write answers and doesn't make assumptions about refund policies.

### Part 2. Website Scarper and Summarizer

- Validates URLs and downloads web pages with Requests for http/https.
- Scrapes readable text and common page noise with BeautifulSoup.
- Breaks down longer text into segments and summarises them with Claude.
- Implements a maximum summary length of 120 words, default is 120 words.

### Part 3. Document-Based AI Agent

- Locates paragraphs in a text based on keywords.
- Saves information related to the conversation history and the name of the user from his session.
- If an arithmetic request is detected, it uses an arithmetic calculator with limited functionality in Python.
- Tells Claude what to answer and what information is missing.

## 4. Technology Stack

The project uses Python, the Anthropic Claude API, Requests for HTTP requests, BeautifulSoup for HTML parsing, Python AST and operator for calculations, Pytest for testing, and environment variables for configuration.

## 5. Design Trade-offs and Failure Handling

- **Rule-based Classification:** Fast and predictable but may miss keyword variations; retain human review.
- **Keyword Retrieval:** Simple and inexpensive but may miss semantic matches; consider embeddings.

- **Chunk-based Summarization:** Handles long pages but may lose context; enforce word limits.
- **Requests and Beautiful Soup:** Lightweight but cannot render JavaScript; consider browser automation.
- **Restricted Calculator:** Prevents arbitrary code execution but supports limited arithmetic; reject unsupported expressions.

## 6. Testing and Operational Considerations

- Automated testing: Passed 20 tests in all three modules: 3 with support; 9 with scraper; 8 with document-agent.
- Manual verification: Tested critical support escalation, classification, knowledge base drafting, webpage extraction and summarizing long content.
- Security: API keys are stored as environment variables and should never be added to git hub.
- Future enhancements: Structured logging, retries, monitoring, expanded test coverage, and advanced document retrieval.